Desenvolvimento Front-End para Web

Universidade Cruzeiro do Sul — Ciência da Computação

TechStore Cadastro

Guia de Implementação

Formulário de Cadastro Full-Stack

HTML + CSS + JavaScript + Supabase + GitHub + Vercel

Prof. Marcos Venícios

1º Semestre — 2026/2

Sumário

1. Visão Geral e Arquitetura
2. Configurando o Supabase (Banco de Dados)
3. Estrutura HTML do Formulário
4. Estilização CSS
5. Integração JavaScript + Supabase
6. Publicando no GitHub
7. Deploy na Vercel
8. Teste e Validação
9. Troubleshooting (Resolução de Problemas)
10. Referência Rápida

1. Visão Geral e Arquitetura

Neste guia, você vai construir um formulário de cadastro completo para a TechStore — uma loja fictícia de tecnologia que temos usado nas aulas anteriores. O formulário será publicado na internet e os dados serão salvos em um banco de dados real.

1.1 Stack Tecnológica

Tecnologia	Função
HTML	Estrutura semântica do formulário (campos, labels, botão)
CSS	Estilização visual moderna com variáveis, transições e responsividade
JavaScript	Captura de dados, validação e envio ao banco via API
Supabase	Banco de dados PostgreSQL na nuvem com API REST automática
GitHub	Repositório remoto para versionamento do código
Vercel	Plataforma de deploy contínuo — publica automaticamente

1.2 Fluxo da Aplicação

```
Navegador (HTML + CSS + JS)
       |
       | fetch / insert
       v
Supabase (PostgreSQL na nuvem)

GitHub (repositório) → Vercel (deploy automático)
                              |
                              v
                      URL pública (HTTPS)
```

O navegador executa o HTML/CSS/JS. Quando o usuário clica em "Cadastrar", o JavaScript envia os dados diretamente ao Supabase via API REST. O código-fonte fica no GitHub, e a Vercel detecta mudanças e publica automaticamente.

2. Configurando o Supabase

O Supabase é um Backend-as-a-Service (BaaS) open-source que oferece um banco PostgreSQL com API REST automática, autenticação e painel visual — tudo sem precisar instalar software.

2.1 Criando a Conta e o Projeto

Passo 1: Acesse **supabase.com** e clique em "Start your project". Você pode criar a conta usando sua conta do GitHub.

Passo 2: Clique em "New Project" e preencha:

- **Nome:** techstore
- **Senha do banco:** Escolha uma senha forte (anote-a, mas não usaremos no código)
- **Região:** South America (São Paulo) — sa-east-1

Passo 3: Aguarde ~2 minutos para o projeto ser criado.

2.2 Criando a Tabela

Vá ao SQL Editor (menu lateral) e cole o seguinte SQL:

```
CREATE TABLE cadastros (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome TEXT NOT NULL,
  email TEXT NOT NULL,
  telefone TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

Coluna	Tipo	Descrição
id	BIGINT	Identificador único, gerado automaticamente
nome	TEXT NOT NULL	Nome completo do cliente (obrigatório)
email	TEXT NOT NULL	E-mail do cliente (obrigatório)
telefone	TEXT	Telefone de contato (opcional)
created_at	TIMESTAMPTZ	Data/hora do cadastro (automática)

2.3 Configurando a Política de Segurança (RLS)

> ⚠️ **IMPORTANTE**
> Sem esta configuração, o formulário NÃO conseguirá salvar dados. O Supabase bloqueia tudo por padrão!

Execute no SQL Editor:

```
-- Habilitar Row Level Security na tabela
ALTER TABLE cadastros ENABLE ROW LEVEL SECURITY;

-- Permitir que qualquer pessoa insira dados
CREATE POLICY "Permitir insercao anonima"
  ON cadastros FOR INSERT TO anon
  WITH CHECK (true);
```

Em produção, as políticas seriam mais restritivas (exigindo autenticação). Para a aula, permitimos inserção anônima pois o foco é o front-end.

2.4 Copiando as Credenciais

Vá em **Settings** → **API** e copie dois valores:

- **Project URL:** https://xxxxxxxxxxxx.supabase.co
- **anon public key:** eyJhbGciOiJIUzI1... (uma string longa)

> 💡 **Segurança da anon key**
> A anon key é pública por design — pode ficar no código front-end. A segurança real é feita pelas políticas de RLS que configuramos acima.

3. Estrutura HTML do Formulário

O formulário usa elementos semânticos do HTML5: <form>, <label>, <input> e <button>. Cada campo tem validação nativa do navegador.

3.1 Estrutura Base

Crie um arquivo chamado index.html e comece com a estrutura base:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>TechStore - Cadastro</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link href="https://fonts.googleapis.com/
    css2?family=Inter:wght@400;500;600;700&display=swap"
    rel="stylesheet">
</head>
<body>
  <!-- Conteúdo aqui -->
</body>
</html>
```

3.2 O Formulário

```
<div class="container">
  <h1>Cadastro TechStore</h1>
  <p class="subtitulo">Cadastre-se para ofertas</p>

  <form id="formCadastro">
    <div class="campo">
      <label for="nome">Nome Completo *</label>
      <input type="text" id="nome" name="nome"
        placeholder="Ex: Maria da Silva" required>
    </div>

    <div class="campo">
      <label for="email">E-mail *</label>
      <input type="email" id="email" name="email"
        placeholder="Ex: maria@email.com" required>
    </div>

    <div class="campo">
      <label for="telefone">Telefone</label>
      <input type="tel" id="telefone"
        name="telefone"
        placeholder="Ex: (11) 99999-9999">
    </div>

    <button type="submit">Cadastrar</button>
```

```
  </form>

  <div id="mensagem"></div>
</div>
```

Elementos-chave:

`for/id:` O atributo for do <label> aponta para o id do <input>, criando uma conexão acessível.

`required:` Marca o campo como obrigatório — o navegador impede o envio se estiver vazio.

`type="email":` O navegador valida automaticamente o formato (precisa ter @ e domínio).

`type="tel":` Em dispositivos móveis, abre o teclado numérico.

4. Estilização CSS

A estilização usa CSS moderno com variáveis (Custom Properties), flexbox para centralização, transições para animações suaves e media queries para responsividade.

4.1 Variáveis CSS (Custom Properties)

Todas as cores ficam centralizadas no :root. Para mudar o visual inteiro, basta alterar os valores aqui:

```
:root {
  --cor-primaria: #1B3A5C;
  --cor-secundaria: #E07B2A;
  --cor-fundo: #F5F0E8;
  --cor-branco: #FFFFFF;
  --cor-texto: #212121;
  --cor-sucesso: #27AE60;
  --cor-erro: #E74C3C;
}
```

4.2 Layout e Centralização

```
body {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: var(--cor-fundo);
}

.container {
  background: var(--cor-branco);
  padding: 2.5rem;
  border-radius: 16px;
  box-shadow: 0 10px 30px rgba(0,0,0,0.08);
  max-width: 450px;
  width: 100%;
}
```

💡 **Flexbox em 3 linhas**

display: flex + justify-content: center + align-items: center = conteúdo centralizado horizontal e verticalmente. Funciona porque o body ocupa 100vh (100% da altura da tela).

4.3 Campos de Input com Estados

```
input {
  width: 100%;
  padding: 0.75rem 1rem;
  border: 2px solid #E0DDD5;
  border-radius: 8px;
```

```
  font-size: 1rem;
  transition: all 0.3s ease;
}

input:focus {
  outline: none;
  border-color: var(--cor-primaria);
  box-shadow: 0 0 0 3px rgba(27,58,92,0.1);
}
```

A propriedade transition: all 0.3s ease faz com que qualquer mudança visual (borda, sombra) aconteça gradualmente em 0.3 segundos, criando um efeito suave e profissional.

4.4 Responsividade

```
@media (max-width: 480px) {
  .container {
    margin: 1rem;
    padding: 1.5rem;
  }
  h1 { font-size: 1.4rem; }
}
```

O @media query aplica estilos diferentes quando a tela é menor que 480px (celular). O container ganha margem lateral e o título diminui.

5. Integração JavaScript + Supabase

O JavaScript é responsável por capturar os dados do formulário e enviá-los ao banco de dados Supabase. Usamos a biblioteca oficial do Supabase via CDN.

5.1 Carregando o Cliente Supabase

Adicione antes do fechamento </body>:

```
<script src="https://cdn.jsdelivr.net/npm/
  @supabase/supabase-js@2"></script>
<script>
const SUPABASE_URL = "https://SEU-PROJETO.supabase.co";
const SUPABASE_ANON_KEY = "SUA-CHAVE-AQUI";

const supabase = window.supabase
  .createClient(SUPABASE_URL, SUPABASE_ANON_KEY);
</script>
```

> ⚠️ **Substitua as credenciais!**
> Cada aluno deve usar as credenciais do próprio projeto Supabase (copiadas na Seção 2.4).

5.2 Capturando o Envio do Formulário

```
const formulario = document.getElementById("formCadastro");

formulario.addEventListener("submit", async (evento) => {
  evento.preventDefault();

  const nome = document.getElementById("nome")
    .value.trim();
  const email = document.getElementById("email")
    .value.trim();
  const telefone = document.getElementById("telefone")
    .value.trim();
```

O que cada linha faz:

`addEventListener("submit", ...)`: Escuta o evento de envio do formulário.

`async`: Marca a função como assíncrona (pode usar await).

`preventDefault()`: Impede o comportamento padrão (recarregar a página).

`.value.trim()`: Pega o texto digitado e remove espaços extras das pontas.

5.3 Enviando Dados ao Supabase

```
  try {
    const { data, error } = await supabase
      .from("cadastros")
      .insert([{ nome, email, telefone }])
      .select();

    if (error) throw error;

    mostrarMensagem("Cadastro realizado!", "sucesso");
    formulario.reset();

  } catch (erro) {
    mostrarMensagem("Erro: " + erro.message, "erro");
  }
});
```

Anatomia da chamada:

- `.from("cadastros")` → seleciona a tabela
- `.insert([{...}])` → insere um registro (array de objetos)
- `.select()` → retorna o registro inserido (confirmação)

5.4 Função de Feedback Visual

```
function mostrarMensagem(texto, tipo) {
  const div = document.getElementById("mensagem");
  div.textContent = texto;
  div.className = "mensagem " + tipo;

  setTimeout(() => {
    div.textContent = "";
    div.className = "";
  }, 5000);
}
```

A função mostra uma mensagem de sucesso (verde) ou erro (vermelha) e a remove automaticamente após 5 segundos usando setTimeout.

6. Publicando no GitHub

O GitHub é uma plataforma para armazenar e versionar código. Vamos usá-lo para guardar o código e conectá-lo à Vercel.

6.1 Upload Manual (Recomendado para a aula)

1. Crie uma conta em **github.com** (se ainda não tiver).
2. Clique em "New Repository" (botão verde).
3. Nome: **`techstore-cadastro`**, marque "Public", clique "Create".
4. Clique em "Add file" → "Upload files".
5. Arraste o **`index.html`** para a área de upload.
6. Clique em "Commit changes".

6.2 Alternativa: Via Linha de Comando

```
git init
git add index.html
git commit -m "Formulário de cadastro TechStore"
git remote add origin https://github.com/
  SEU-USUARIO/techstore-cadastro.git
git push -u origin main
```

> 💡 **Para esta aula**
> O upload manual pelo site funciona perfeitamente. O Git pela linha de comando é mais poderoso para projetos maiores, mas não é necessário agora.

7. Deploy na Vercel

A Vercel é uma plataforma de deploy contínuo. Ela conecta ao seu repositório GitHub e publica automaticamente a cada mudança no código.

7.1 Passo a Passo

1. Acesse **vercel.com** e clique em "Sign Up".
2. Faça login com sua conta do GitHub.
3. Clique em "Add New..." → "Project".
4. Encontre e selecione o repositório `techstore-cadastro`.
5. Clique em "Deploy" — não precisa configurar nada!
6. Aguarde ~10 segundos.

> 🎉 **Resultado!**
> Sua URL estará em: https://techstore-cadastro-seunome.vercel.app

7.2 Deploy Contínuo

A partir de agora, qualquer mudança que você fizer no repositório GitHub será automaticamente publicada pela Vercel. Se você atualizar o index.html no GitHub, em poucos segundos o site será atualizado.

8. Teste e Validação

8.1 Checklist de Validação

#	Verificação	Status
1	Formulário abre sem erros no navegador	[ ]
2	Campos de nome e e-mail são obrigatórios	[ ]
3	E-mail inválido é rejeitado pelo navegador	[ ]
4	Ao enviar, mensagem verde de sucesso aparece	[ ]
5	Dados aparecem no painel do Supabase	[ ]
6	Formulário limpa após envio com sucesso	[ ]
7	Layout funciona no celular (responsivo)	[ ]
8	Site acessível pela URL da Vercel	[ ]

8.2 Teste Cruzado

Compartilhe sua URL com um colega. Peça para ele preencher seu formulário. Verifique no painel do Supabase se os dados chegaram corretamente. Depois, faça o mesmo com o formulário do colega.

9. Troubleshooting

Problemas comuns e suas soluções:

Problema: "Erro ao cadastrar" ao enviar o formulário

Causa mais provável: Política de RLS não foi criada.

Solução: Execute o SQL da Seção 2.3 no SQL Editor do Supabase.

Problema: Página em branco ao abrir o index.html

Causa mais provável: Erro de sintaxe no HTML ou JS.

Solução: Abra o Console do navegador (F12 → Console) e veja a mensagem de erro.

Problema: createClient is not a function

Causa mais provável: A tag <script> do CDN do Supabase está incorreta ou faltando.

Solução: Verifique se o CDN está carregado ANTES do seu <script>.

Problema: Vercel mostra erro 404

Causa mais provável: O arquivo não se chama index.html.

Solução: Renomeie para index.html (tudo minúsculo) e faça commit novamente.

Problema: Dados não aparecem no Supabase

Causa mais provável: Credenciais incorretas no código.

Solução: Verifique se a URL e a anon key correspondem ao seu projeto (Settings → API).

Problema: Formulário recarrega a página ao enviar

Causa mais provável: Faltou o evento.preventDefault() no listener.

Solução: Verifique se preventDefault() é a primeira linha dentro do callback do addEventListener.

10. Referência Rápida

10.1 Conceitos HTML

Elemento / Atributo	Função
`<form>`	Container que agrupa campos de formulário
`<label for="...">`	Rótulo acessível conectado ao input
`<input type="...">`	Campo de entrada (text, email, tel, password)
`required`	Torna o campo obrigatório
`placeholder`	Texto-dica que aparece dentro do campo vazio

10.2 Conceitos CSS

Propriedade	O que faz
`:root { --var: val }`	Define variáveis CSS globais
`display: flex`	Ativa layout flexbox no container
`transition: all 0.3s`	Animação suave nas mudanças de propriedade
`:focus / :hover`	Pseudo-classes de estado do elemento
`@media (max-width)`	Media query para responsividade

10.3 Conceitos JavaScript

Código	O que faz
`getElementById()`	Busca elemento pelo atributo id
`addEventListener()`	Escuta um evento (submit, click, etc.)
`preventDefault()`	Impede comportamento padrão
`async / await`	Espera operações assíncronas
`try / catch`	Tratamento de erros
`.from().insert().select()`	Cadeia de inserção no Supabase

— Fim do Guia de Implementação —